PAP 2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Dieses Dokument ist von mir (FEZ) selbstständig geschreiben und verfasst worden. Viele Funktionen sind von anderen übernommen und abgewandelt worden. Da der Code in VS Code geschrieben wird, wird teilweise auf den MS Copilot (K.I. Modell) zurückgegriffen. Dieser wird unterstützend und nicht generierend benutzt. Ich entschuldige mit hiermit auch formal für das unglaublich schlimme Denglisch in den Kommentaren und der Dokumentation.

Vorlage zum bestimmen aller im Praktikum benötigten Formel und intigriertem Latex Export für eine effizientere Nutzung.

Diese Version ist noch nicht aktiell. Daher führen viele Überschriften in Leere Kapitel. (Stand 07.03.2026)

Inhaltsverzeichnis für den Papulator

- Definition der Versuchsvariablen
- Import aller genutzen Libaries
- Funktionen der Fehlerrechnung
  - Runden signifikanter Stellen
  - Gaussische Fehlerfortpflanzung
  - Berechnung von Messwert und Fehler
  - Signifikante Standartabweichung zweier Messwerte
- Messdatenimporte
  - Patterns
  - Import von Experimentdaten
- Latex Export
  - Export von Gleichungen
  - Export von Einzelergebnissen
  - Export von Messreihen (Tabelle))
- Plotting
- Ergebnisse der Aufgaben

```
    # Calculate the results for each row of data
    results = np.zeros((data.shape[0],2))
    for n in np.arange(0,data.shape[0]):
        results[n,:] = calc_with_err(f, err_abs, exp_data[n,:])
        # print(calc_with_err(f, err_abs, exp_data[n]))

    # if (len(results) < 10):
    #     print("Results:")
    #     print(results)

    # Substitutes 0 for the uncertainty of the parameters without error, so it doesnt show up in
    for p in params_without_error:
        absolut_err = absolut_err.subs('thisWillTurnDelta'+p.name,0)
    for p in params_without_error:
        relativ_err = relativ_err.subs('thisWillTurnDelta'+p.name,0)

    if print_formula:
    # Wiedergabe des Latex-Codes
        function = sp.simplify(function,symbols = params, rational= True)
        function = sp.separatevars(function)

        print("gegebene Funktion:")
        function_to_latex(function)

        print("Formel des absoluten Fehlers der gegebenen Funktion:")
        function_to_latex(absolut_err)

        print("Formel des relativen Fehlers der gegebenen Funktion:")
        function_to_latex(relativ_err)

    return(results)


def calc2(function, params,data, params_without_error = [], p_latex:bool = True):
    length = 1
    for e in data:
        if(np.shape(np.array(e)) != ()):
            length = np.shape(np.array(e))[0]
            break
    data_2d = np.zeros((length,len(data)))
    for i in range (0,len(data)):
        data_2d[:,i] = data[i]
    return(calc_everything(function,params, data_2d,params_without_error = params_without_error,
```

Plotting

Ergebnisse der Aufgaben

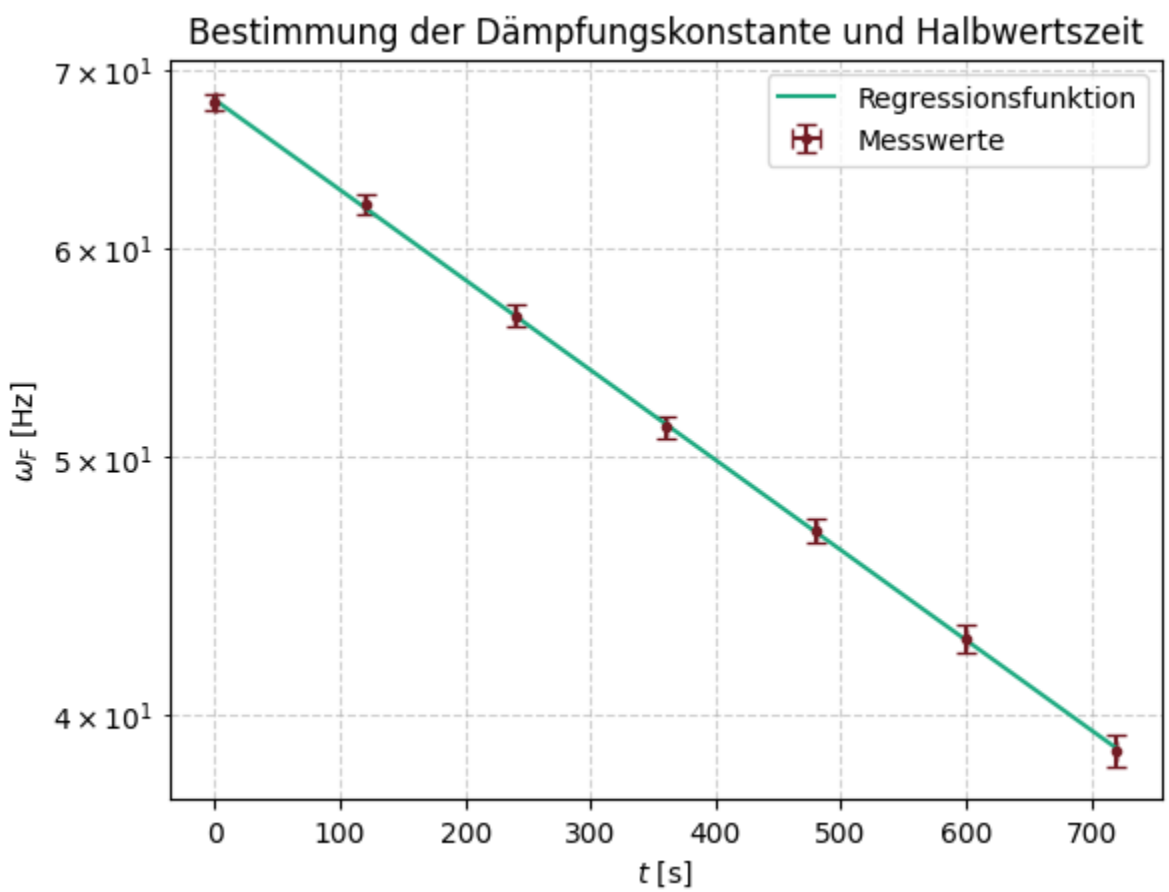
Aufgabe 1

In [104…

```
aufgabe = "1"
pfad = "../img/"

# ------------------------
#   Messdaten
# ------------------------

# 1/min
freq = np.array([650, 595, 540, 490, 448, 408, 370])
delta_freq = 5

# Messzeit in Sekunde
t = np.array([0, 120, 240, 360, 480, 600, 720])
err_t = 0.3

# Umrechung in Kreisfrequenz und Einheiten in pro sekunde
omega_F = freq * 2* np.pi /60
err_omega = delta_freq * 2 * np.pi /60


# ------------------------
#   Ausgleichsgerade
# ------------------------
# linearer Regression; Hier exponential Funktion, da wir logarithmiosch Skalieren
# daher vermutlich formal nicht wirklich lineare Regression, aber wird eine visuelle Gerade

def exp(x,a,k):
    return(a*np.exp(-k*x))

# Ohne p0 berechnet sich der Fehler zu 0. Fix Idee kam durech K.I.
popt, pcov = curve_fit(exp, t, omega_F, p0=[omega_F[0], 1e-3], sigma=err_omega, absolute_sigma=Tr

# Dämpfung (Achsenabschnitt der Ordinate)
damp = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))
print(f"Dämpfungskonstante k = {damp[2]} 1/s")

# Berechnung der Halbwertszeit
hl_val = np.log(2)/float(damp[0])
hl_err = np.log(2)*float(damp[1])/(float(damp[0])**2)

halb_t = round_sig_digs(hl_val, hl_err)
print(f"Halbwertszeit T_h = {halb_t[2]} s")


# ------------------------
#   Plotting
# ------------------------

plt.errorbar(
    t,
    omega_F,
    xerr=err_t, # basically unnötig in der größen Ordnung, aber mir auch egal
    yerr=err_omega,
    fmt=".",
    color=weinrot,
    capsize=3.5,
    label="Messwerte",
    )

plt.plot(
    t,
```

```
    exp(t,*popt),
    color=hellgruen,
    label= "Regressionsfunktion",
    )

plt.yscale("log")
plt.xlabel("$t \\ [\\mathrm{s}]$")
plt.ylabel("$\\omega_F \\ [\\mathrm{Hz}]$")
plt.title("Bestimmung der Dämpfungskonstante und Halbwertszeit")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()

# plt.text(450, 60, "Dies wäre mein Text")

plt.savefig(f'{pfad}Damping_{aufgabe}.pdf', dpi=300)
plt.show()
```

```
Dämpfungskonstante k = (7.82 $\pm$ 0.16) e-4 1/s
Halbwertszeit T_h = 9e+2 $\pm$ 2e+1 s
```



Aufgabe 3

In [105…

```
# ------------------------
#   Messdaten
# ------------------------

# Gemessene Frequenzen [1/min]
## Start Frquenzen
freq_start_1 = np.array([600] * 4)
freq_start_2 = np.array([450] * 4)
freq_start_3 = np.array([300] * 4)

## Zusammenfassen und umrechnen in Kreisfrequentz [1/sek]
freq_s = np.array([freq_start_1, freq_start_2, freq_start_3]) * 2 * np.pi / 60
```

```
"""
**Erster Indize** : ändert Startfrequenz

**Zweiter Indize** : ändert Massen-Zustand (hier identisch)
"""



## End Frequenzen
freq_end_1 = np.array([520, 540, 528, 490])
freq_end_2 = np.array([405, 415, 395, 335])
freq_end_3 = np.array([275, 280, 250, 240])

## Zusammenfassen und umrechnen in Kreisfrequentz [1/sek]
freq_e = np.array([freq_end_1, freq_end_2, freq_end_3]) * 2 * np.pi / 60
"""
**Erster Indize** : ändert Endfrequenz (von 0 bis 2) [Dazugehörende Startfrequenz]
- 0 : 600 rpm
- 1 : 450 rpm
- 2 : 300 rpm

**Zweiter Indize** : ändert Massen-Zustand (von 0 bis 3)
- 0 : 1m, 20cm
- 1 : 2m, 20cm
- 2 : 2m, 15cm
- 3 : 1m, 15cm
"""

# Fehler auch in Kreisfrequenz und Einheiten anpassen
err_freq = 5 * 2 * np.pi / 60

# Gemessene Zeiten [s]
time_1 = np.array([79.7, 42.2, 54.5, 105.8])
time_2 = np.array([63.7, 32.8, 43.6, 81.4])
time_3 = np.array([44.6, 23.1, 28.6, 55.5])

T_p = np.array([time_1, time_2, time_3])
"""
**Erster Indize** : ändert Endfrequenz (von 0 bis 2) [Dazugehörende Startfrequenz]
- 0 : 600 rpm
- 1 : 450 rpm
- 2 : 300 rpm

**Zweiter Indize** : ändert Massen-Zustand (von 0 bis 3)
- 0 : 1m, 20cm
- 1 : 2m, 20cm
- 2 : 2m, 15cm
- 3 : 1m, 15cm
"""

err_T_p = 0.3

# Und ja, die Markdownkommentare war ich (keine K.I.), in VS Code kann man so über die Variable h
```

EINSCHUG: Vergleich der theoretischen und der gemessenen Endfrequenz

In [106…

```
f_a_sym, T_p_sym, delta_sym = sp.symbols('f_\\text{a}, T_\\text{p}, \\delta')
params = [f_a_sym, T_p_sym, delta_sym]

f_e = f_a_sym * sp.exp(-T_p_sym * delta_sym)
```

```python
freq_e_vgl = np.array([freq_end_1, freq_end_2, freq_end_3]) /60
freq_s_vgl = np.array([freq_start_1, freq_start_2, freq_start_3]) /60
err_gem = 5 /60

rows = []
f_end_theo = []
err_f_end_theo = []
for n in range(3):
    for i in range(4):

        f_gem = freq_e_vgl[n][i]

        values = np.array([
            freq_s_vgl[n][0], err_gem,
            T_p[n][i], err_T_p,
            damp[0], damp[1]
        ])

        f_theo = calc2(f_e, params, values, [], False)

        f_end_theo.append(f_theo[0][0] * 2 * np.pi)
        err_f_end_theo.append(f_theo[0][1] * 2 * np.pi)

        # Abweichung
        sigma = abs(f_gem - f_theo[0][0]) / np.sqrt(err_gem**2 + f_theo[0][1]**2)

        # Als String speichern
        meas_str = f"{f_gem:.2f} \\pm {err_gem:.2f}"
        theo_str = f"{f_theo[0][0]:.2f} \\pm {f_theo[0][1]:.2f}"
        sigma_str = f"{sigma:.2f}"

        rows.append([meas_str, theo_str, sigma_str])


# Panas für latex Export
df_theo_vgl = pd.DataFrame(
    rows,
    columns=[
        "f_{\\text{gem}} \\, [\\mathrm{s^{-1}}]",
        "f_{\\text{theo} \\, [\\mathrm{s^{-1}}]",
        "\\text{St.Abw. } \\sigma"
    ]
)

latex_table = df_theo_vgl.to_latex(
    index=False,
    escape=False,
    column_format="LLL",
    caption="Vergleich gemessener und theoretischer Endfrequenzen",
    label="tab:freq_compare"
)

print(f_end_theo)

print(latex_table)
```

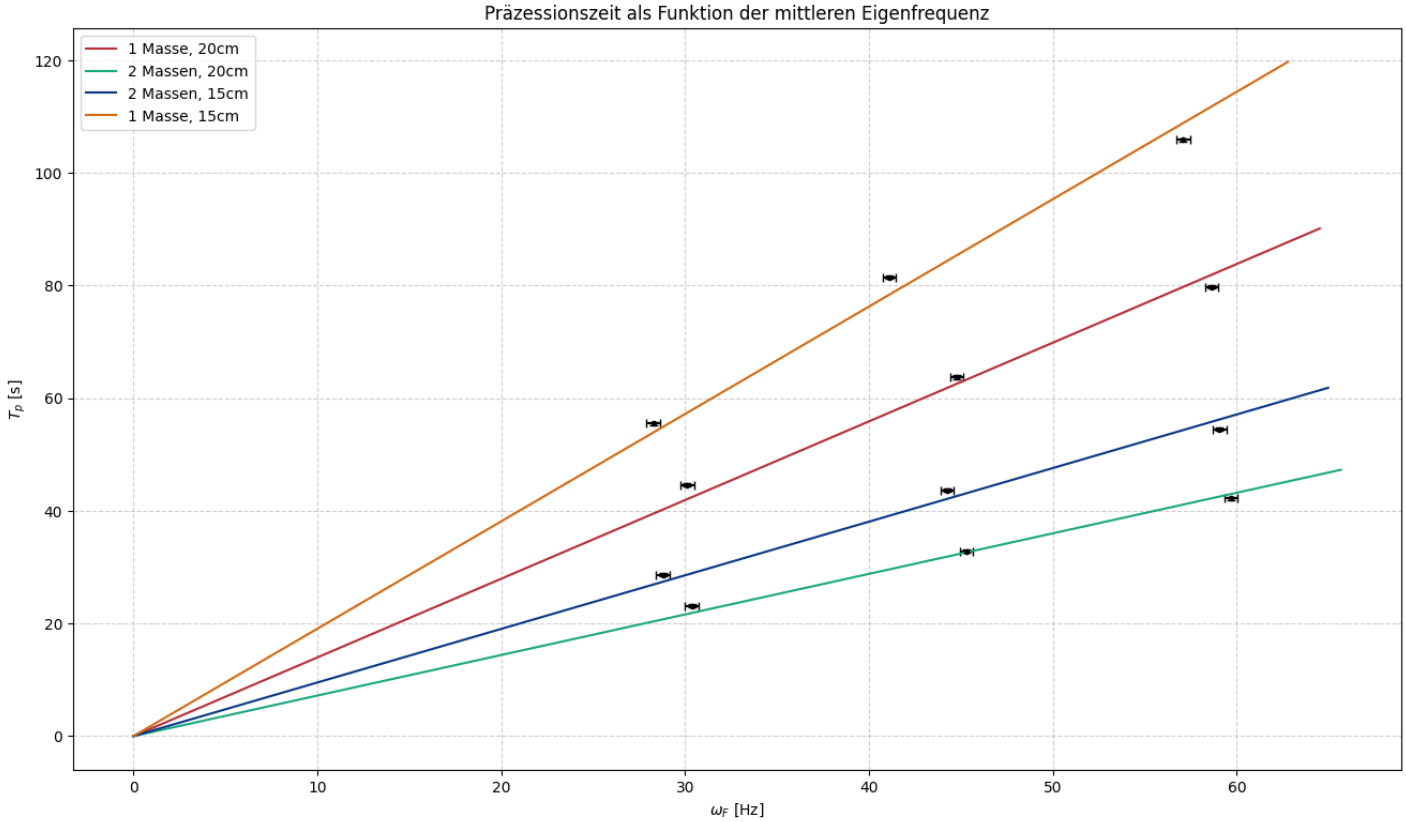
```
[np.float64(59.03537021226212), np.float64(60.792216300503675), np.float64(60.21028346495402), n
p.float64(57.84265963416527), np.float64(44.83399582146718), np.float64(45.93055064033751), np.fl
oat64(45.54427305089058), np.float64(44.217705551044794), np.float64(30.33911423429569), np.float
64(30.85351797955085), np.float64(30.7211019634892), np.float64(30.08160871425159)]
\begin{table}
\caption{Vergleich gemessener und theoretischer Endfrequenzen}
\label{tab:freq_compare}
\begin{tabular}{LLL}
\toprule
f_{\text{gem}} \, [\mathrm{s^{-1}]} & f_{\text{theo} \, [\mathrm{s^{-1}]} & \text{St.Abw. } \sigm
a \\
\midrule
8.67 \pm 0.08 & 9.40 \pm 0.08 & 6.34 \\
9.00 \pm 0.08 & 9.68 \pm 0.08 & 5.81 \\
8.80 \pm 0.08 & 9.58 \pm 0.08 & 6.76 \\
8.17 \pm 0.08 & 9.21 \pm 0.08 & 9.09 \\
6.75 \pm 0.08 & 7.14 \pm 0.08 & 3.34 \\
6.92 \pm 0.08 & 7.31 \pm 0.08 & 3.38 \\
6.58 \pm 0.08 & 7.25 \pm 0.08 & 5.73 \\
5.58 \pm 0.08 & 7.04 \pm 0.08 & 12.68 \\
4.58 \pm 0.08 & 4.83 \pm 0.08 & 2.12 \\
4.67 \pm 0.08 & 4.91 \pm 0.08 & 2.09 \\
4.17 \pm 0.08 & 4.89 \pm 0.08 & 6.20 \\
4.00 \pm 0.08 & 4.79 \pm 0.08 & 6.82 \\
\bottomrule
\end{tabular}
\end{table}
```

In [107…

```python
aufgabe = "3"
pfad = "../img/"

# ------------------------
#   Graphen erstellen
# ------------------------

# Jedem Plot ein Label geben. Reihenfolge muss identisch zu dem Format oben sein!
labels = [
    "1 Masse, 20cm",
    "2 Massen, 20cm",
    "2 Massen, 15cm",
    "1 Masse, 15cm"
]

# Farben müssen halt mega cool aussehen, sonst ist das Protokoll nur mid
colors = [
    rot,
    hellgruen,
    blau,
    orange,
    ]

plt.figure(figsize=(16,9))

steig_3 = []
err_steig_3 = []
for j in range(4): # 4 Massenzustände

    x = []
    y = []

    for i in range(3): # 3 Startfrequenzen
```

```python
        avg_freq = (freq_s[i, j] + freq_e[i, j]) / 2
        x.append(avg_freq)
        y.append(T_p[i, j])

    x = np.array(x)
    y = np.array(y)


    # Regression durch Ursprung
    m = np.sum(x*y) / np.sum(x**2)

    res = y - m*x
    n = len(x)

    sigma_m = np.sqrt(np.sum(res**2) / ((n-1) * np.sum(x**2)))

    steig_3.append(m)
    err_steig_3.append(sigma_m)

    # Kam auf linspace durch einen kommunitonen ('L. Hafner')
    x_fit = np.linspace(0, max(x)*1.1, 1000)
    y_fit = m * x_fit

    plt.errorbar(
        x, y,
        xerr=err_freq / np.sqrt(2), # Da Mittlweret auch std
        yerr=err_T_p,
        fmt='.',
        # color=colors[j],
        color="black",
        capsize=3
        )

    plt.plot(
        x_fit,
        y_fit,
        color=colors[j],
        label=f"{labels[j]}",
        )


for i in range(4):
    st_3 = round_sig_digs(steig_3[i], err_steig_3[i])
    print(f"Steigung {i + 1} = {st_3[2]}")

# ------------------------
#   Erstellung des Plots
# ------------------------

plt.xlabel("$\\omega_F \\ [\\mathrm{Hz}]$")
plt.ylabel("$T_p \\ [\\mathrm{s}]$")
plt.title('Präzessionszeit als Funktion der mittleren Eigenfrequenz')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()

plt.savefig(f'{pfad}tp_gegen_omegaAVG{aufgabe}.pdf', dpi=300)
plt.show()
```

```
Steigung 1 = 1.4 $\pm$ 0.03
Steigung 2 = (7.20 $\pm$ 0.13) e-1
Steigung 3 = (9.52 $\pm$ 0.23) e-1
Steigung 4 = 1.9 $\pm$ 0.04
```



Mit theoretischer Zeit selber Plot

In [108…

```
# aufgabe = "3_alt"
# pfad = "../img/"

# # Wieder ein 2d-Array
# f_end_theo = np.array(f_end_theo).reshape(3, 4)



# # ------------------------
# #   Graphen erstellen
# # ------------------------

# # Jedem Plot ein Label geben. Reihenfolge muss identisch zu dem Format oben sein!
# labels = [
#     "1 Masse, 20cm",
#     "2 Massen, 20cm",
#     "2 Massen, 15cm",
#     "1 Masse, 15cm"
# ]

# # Farben müssen halt mega cool aussehen, sonst ist das Protokoll nur mid
# colors = [
#     rot,
#     hellgruen,
#     blau,
#     orange,
#     ]

# plt.figure(figsize=(16,9))

# steig_3 = []
# err_steig_3 = []
# for j in range(4): # 4 Massenzustände
```

```
#     x = []
#     y = []

#     for i in range(3): # 3 Startfrequenzen
#         avg_freq = (freq_s[i, j] + f_end_theo[i, j]) / 2
#         x.append(avg_freq)
#         y.append(T_p[i, j])

#     x = np.array(x)
#     y = np.array(y)


#     # Regression durch Ursprung
#     m = np.sum(x*y) / np.sum(x**2)

#     res = y - m*x
#     n = len(x)

#     sigma_m = np.sqrt(np.sum(res**2) / ((n-1) * np.sum(x**2)))

#     steig_3.append(m)
#     err_steig_3.append(sigma_m)

#     # Kam auf linspace durch einen Kommunitonen ('L. Hafner')
#     x_fit = np.linspace(0, max(x)*1.1, 1000)
#     y_fit = m * x_fit

#     plt.errorbar(
#         x, y,
#         xerr=err_freq / np.sqrt(2), # Da Mittlweret auch std
#         yerr=err_T_p,
#         fmt='.',
#         # color=colors[j],
#         color="black",
#         capsize=3
#         )

#     plt.plot(
#         x_fit,
#         y_fit,
#         color=colors[j],
#         label=f"{labels[j]}",
#         )


# for i in range(4):
#     st_3 = round_sig_digs(steig_3[i], err_steig_3[i])
#     print(f"Steigung {i + 1} = {st_3[2]}")

# # ------------------------
# #   Erstellung des Plots
# # ------------------------

# plt.xlabel("$\\omega_F \\ [\\mathrm{Hz}]$")
# plt.ylabel("$T_p \\ [\\mathrm{s}]$")
# plt.title('Präzessionszeit als Funktion der mittleren Eigenfrequenz')
# plt.grid(True, which="both", linestyle="--", alpha=0.6)
# plt.legend()

# plt.savefig(f'{pfad}tp_gegen_omegaAVG{aufgabe}.pdf', dpi=300)
# plt.show()
```

In [109…

```python
# ------------------------
#   Messwerte setzen
# ------------------------

# Masse eines Gewichtes von kg in g
m = 0.00985 * 1000
m_ar = np.array([m, 2*m, 2*m, m])

# Heidelberger Ortskonstante für g (g_hd und err_hd )

# Abstände l von cm in m
l = np.array([20, 20 , 15, 15]) / 100
err_l = 0.1 / 100

# Steigungen aus Teil davor (Massen und Strecken abhänig)


# ------------------------
#   Berechnung von I_z
# ------------------------

m_sym, g_sym, l_sym, steig_sym, pi_sym= sp.symbols('m, g, l, s, \\pi')

params = [g_sym, l_sym, steig_sym, m_sym, pi_sym]
params_without_error = [m_sym, pi_sym]


I_z = (m_sym * g_sym * l_sym * steig_sym)/(2 * pi_sym)

tr_momente = []
err_tr_momente = []
n = 0


rows = []
for n in range(4):

    values = np.array([
        g_hd, err_g_hd,
        l[n], err_l,
        steig_3[n], err_steig_3[n],
        m_ar[n],
        np.pi,
    ])

    res = calc2(I_z, params, values, params_without_error, p_latex=False)

    tr_momente.append(res[0][0])
    err_tr_momente.append(res[0][1])

    # ------------------------------
    #   Latex Tabelle erstellen
    # ------------------------------

    # Anzahl der Massen
    n_mass = int(m_ar[n] / m)

    #  Müssen strings sein lol (20 minuten meines lebens daran gesucht)
    l_str = f"{l[n]:.3f} \\pm {err_l:.3f}"
    steig_str = f"{steig_3[n]:.3f} \\pm {err_steig_3[n]:.3f}"
```

```
    I_str = f"{tr_momente[n]:.2f} \\pm {err_tr_momente[n]:.2f}"

    rows.append([n_mass, l_str, steig_str, I_str])


df = pd.DataFrame(
    rows,
    columns=[
        "N",
        "l",
        "s",
        "I_z"
    ]
)

latex_table = df.to_latex(
    index=False,
    escape=False,
    column_format="LLLL",
    caption="Bestimmung des Trägheitsmoments $I_z$",
    label="tab:traegheitsmomente"
)

print(latex_table)
```

```
\begin{table}
\caption{Bestimmung des Trägheitsmoments $I_z$}
\label{tab:traegheitsmomente}
\begin{tabular}{LLLL}
\toprule
N & l & s & I_z \\
\midrule
1 & 0.200 \pm 0.001 & 1.397 \pm 0.032 & 4.30 \pm 0.10 \\
2 & 0.200 \pm 0.001 & 0.720 \pm 0.013 & 4.43 \pm 0.08 \\
2 & 0.150 \pm 0.001 & 0.952 \pm 0.023 & 4.39 \pm 0.11 \\
1 & 0.150 \pm 0.001 & 1.906 \pm 0.043 & 4.40 \pm 0.10 \\
\bottomrule
\end{tabular}
\end{table}
```

In [110…

```
# ------------------------
#   Mittelwert von I_z
# ------------------------

trae_mean = np.mean(tr_momente)
err_trae_mean = np.sqrt((np.mean(err_tr_momente))**2 + (np.std(tr_momente))**2)

res_to_p = round_sig_digs(trae_mean, err_trae_mean)

print(res_to_p[0])
```

```
4.38
```

Noch ein einschub

In [111…

```
print(steig_3[1])
```

```
0.7199586268181922
```

Aufgabe 5

Bestimmung der Frequenz und Periodendauer für 10 Farbwechsel

In [112…

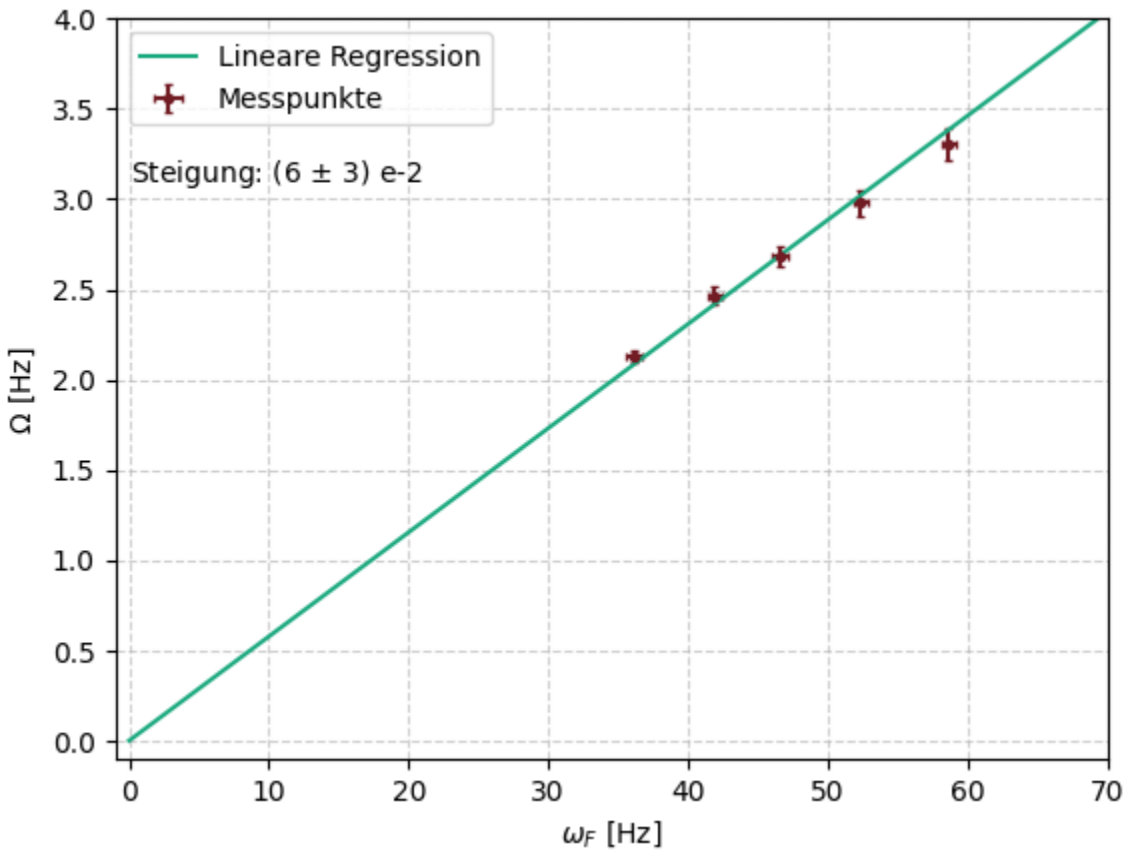
```
iz = np.array([trae_mean, err_trae_mean])
```

In [119…

```
aufgabe = "5b_alt"
pfad = "../img/"

# ------------------------
#   Messdaten
# ------------------------

# Frequenzen
frq = np.array([560, 500, 445, 400, 345])
err_frq = 5

# Umrechnen Kreisfrequenz in Hz
omega_f = frq * 2 * np.pi /60
err_omega_f = err_frq * 2 * np.pi /60

# Zeit für 10 Schwinungen in Sekunden
t = np.array([19.0, 21.1, 23.4, 25.5, 29.5])
err_t = 0.5

# Omega 20pi / t
omega_cap = (20 * np.pi) / t
err_omega_cap = (20 * np.pi) / (t**2) * err_t


# ------------------------
#   Erstellung des Plots
# ------------------------

# Durch den Ursprung laufen lassen; Umsetzungsidee kam von 'Luca Hafner' :)
def linear(x,a):
    return a*x
x = np.linspace(0, 70, 10000)

popt, pcov = curve_fit(linear, omega_f, omega_cap, sigma=omega_cap, absolute_sigma=True)

steig = round_sig_digs(popt[0], np.sqrt(pcov[0,0]))
# absch = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))

print(f"Steigung: {steig[2]} \n", f"y-Achsenabschnitt: {absch[2]}")

plt.xlim([-1, 70])
plt.ylim([-0.1, 4])

plt.errorbar(
    omega_f,
    omega_cap,
    err_omega_cap,
    err_omega_f,
    fmt='.',
    color=weinrot,
    capsize=1.5,
    label="Messpunkte")
```

```
plt.plot(
    x,
    linear(x,*popt),
    color=hellgruen,
    label="Lineare Regression")


plt.xlabel('$\\omega_F \\ [\\mathrm{Hz}]$')
plt.ylabel('$\\Omega \\ [\\mathrm{Hz}]$')
plt.title('')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
# plt.text(340, 3, f"Steigung: {steig[0]} $\\pm$ {steig[1]} \n y-Achsenabschnitt: {absch[0]} $\\p
plt.text(0, 3.1, f"Steigung: {steig[2]}")

plt.savefig(f'{pfad}Omega_gegen_omegaF{aufgabe}.pdf')
plt.show()
```

```
Steigung: (6 $\pm$ 3) e-2
 y-Achsenabschnitt: (2.8 $\pm$ 0.4) e-1
```

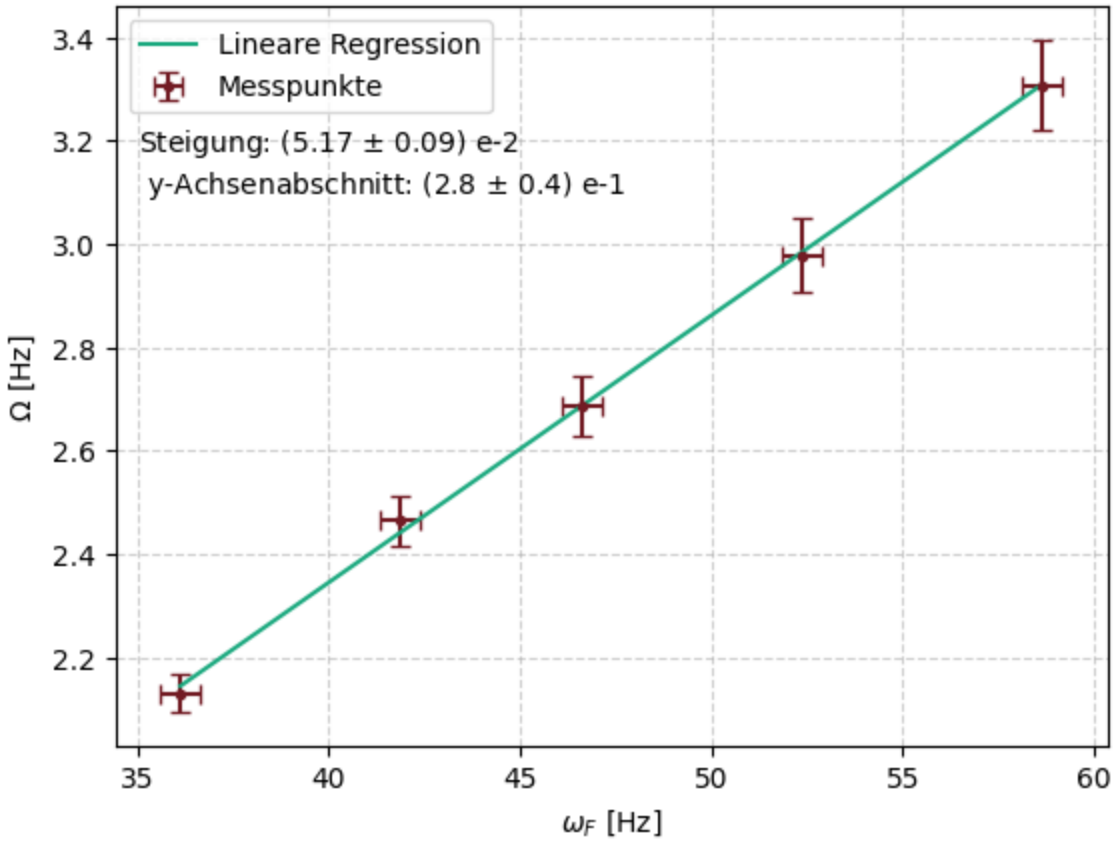


In [120…

```
aufgabe = "5_alt"
pfad = "../img/"

# ------------------------
#   Messdaten
# ------------------------

# Frequenzen
frq = np.array([560, 500, 445, 400, 345])
err_frq = 5

# Umrechnen Kreisfrequenz in Hz
omega_f = frq * 2 * np.pi /60
err_omega_f = err_frq * 2 * np.pi /60
```

```python
# Zeit für 10 Schwinungen in Sekunden
t = np.array([19.0, 21.1, 23.4, 25.5, 29.5])
err_t = 0.5

# Omega 20pi / t
omega = (20 * np.pi) / t
err_omega = (20 * np.pi) / (t**2) * err_t


# ------------------------
#   Erstellung des Plots
# ------------------------

def linear(x,a,b):
    return a*x+b

popt, pcov = curve_fit(linear, omega_f, omega)

steig = round_sig_digs(popt[0], np.sqrt(pcov[0,0]))
absch = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))

print(f"Steigung: {steig[2]} \n", f"y-Achsenabschnitt: {absch[2]}")


plt.errorbar(
    omega_f,
    omega,
    err_omega,
    err_omega_f,
    fmt='.',
    color=weinrot,
    capsize=3.5,
    label="Messpunkte")

plt.plot(
    omega_f,
    linear(omega_f,*popt),
    color=hellgruen,
    label="Lineare Regression")


plt.xlabel('$\\omega_F \\ [\\mathrm{Hz}]$')
plt.ylabel('$\\Omega \\ [\\mathrm{Hz}]$')
plt.title('')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
# plt.text(340, 3, f"Steigung: {steig[0]} $\\pm$ {steig[1]} \n y-Achsenabschnitt: {absch[0]} $\\p
plt.text(35, 3.1, f"Steigung: {steig[2]} \n y-Achsenabschnitt: {absch[2]}")

plt.show()
```

```
Steigung: (5.17 $\pm$ 0.09) e-2
 y-Achsenabschnitt: (2.8 $\pm$ 0.4) e-1
```



In [121…

```
Iz_s, steig_sym = sp.symbols('I_z, \\left(\\frac{\\Omega}{\\omega_F}\\right)')
params = [Iz_s, steig_sym]

Ix = Iz_s/((1/steig_sym) - 1) + Iz_s

values = np.array([
    iz[0], iz[1],
    float(steig[0]), float(steig[1])
])

calc2(Ix, params, values, [])
```

gegebene Funktion:
Die gegebene Funktion lautet:

$$-\frac{I_z}{\left(\frac{\Omega}{\omega_F}\right) - 1}$$

Hier ist der dazugehörige Latex code:

```
- \frac{I_{z}}{\left(\frac{\Omega}{\omega_F}\right) - 1}
```

Kopieren

Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\sqrt{\frac{1}{\left(\left(\frac{\Omega}{\omega_F}\right) - 1\right)^4}\left(I_z^2\Delta\left(\frac{\Omega}{\omega_F}\right)^2 + \Delta I_z^2\left(\left(\frac{\Omega}{\omega_F}\right) - 1\right)^2\right)}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{1}{\left(\left(\frac{\Omega}{\omega_F}\right) - 1\right)^{4}}
\left(I_{z}^{2} \Delta \left(\frac{\Omega}{\omega_F}\right)^{2} + \Delta I_{z}^{2}
\left(\left(\frac{\Omega}{\omega_F}\right) - 1\right)^{2}\right)}
```

Kopieren

Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\sqrt{\frac{\Delta\left(\frac{\Omega}{\omega_F}\right)^2}{\left(\left(\frac{\Omega}{\omega_F}\right)-1\right)^2}+\frac{\Delta I_z^2}{I_z^2}}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{\Delta \left(\frac{\Omega}{\omega_F}\right)^{2}}
{\left(\left(\frac{\Omega}{\omega_F}\right) - 1\right)^{2}} + \frac{\Delta I_{z}^{2}}
{I_{z}^{2}}}
```

Kopieren

Out[121…

```
array([[4.61673054, 0.11640179]])
```

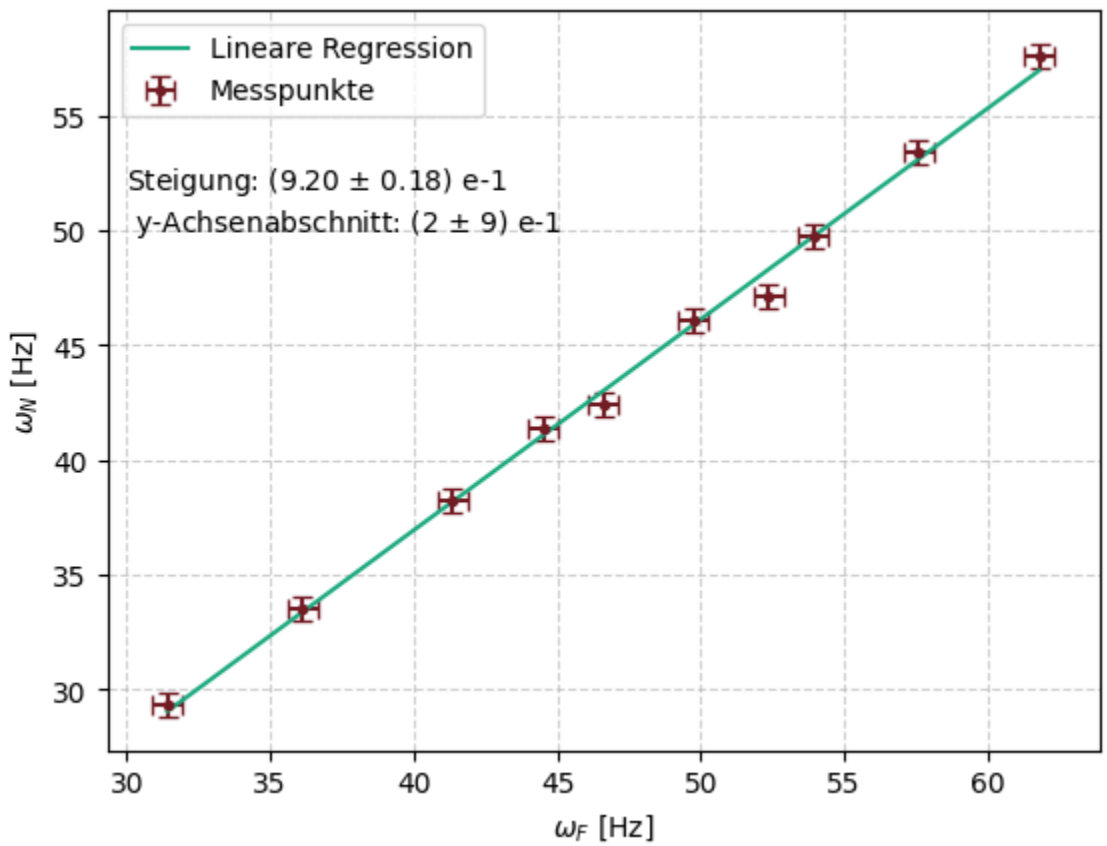
Bestimmung von I_x über Nutation

In [122…

```python
aufgabe = "6b_alt"

# Einheit in Hz
omega_n = np.array([550, 510, 475, 450, 440, 405, 395, 365, 320, 280]) * 2 * np.pi /60
omega_f_2 = np.array([590, 550, 515, 500, 475, 445, 425, 395, 345, 300]) * 2 * np.pi /60

err_omega = 5 * 2 * np.pi /60

# ------------------------
#   Erstellung des Plots
# ------------------------

def linear(x,a,b):
    return a*x + b

popt, pcov = curve_fit(linear, omega_f_2, omega_n, sigma=err_omega, absolute_sigma=True)

steig_2 = round_sig_digs(popt[0], np.sqrt(pcov[0,0]))
absch_2 = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))

print(f"Steigung: {steig_2[2]} \n", f"y-Achsenabschnitt: {absch_2[2]}")


plt.errorbar(
    omega_f_2,
    omega_n,
    err_omega,
    err_omega,
    fmt='.',
    color=weinrot,
```

```
    capsize=3.5,
    label="Messpunkte")

plt.plot(
    omega_f_2,
    linear(omega_f_2,*popt),
    color=hellgruen,
    label="Lineare Regression")


plt.xlabel('$\\omega_F \\ [\\mathrm{Hz}]$')
plt.ylabel('$\\omega_N \\ [\\mathrm{Hz}]$')
plt.title('')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.text(30, 50, f"Steigung: {steig_2[2]} \n y-Achsenabschnitt: {absch_2[2]}")

plt.savefig(f'{pfad}omegaN_gegen_omegaF{aufgabe}.pdf')
plt.show()
```

```
Steigung: (9.20 $\pm$ 0.18) e-1
 y-Achsenabschnitt: (2 $\pm$ 9) e-1
```

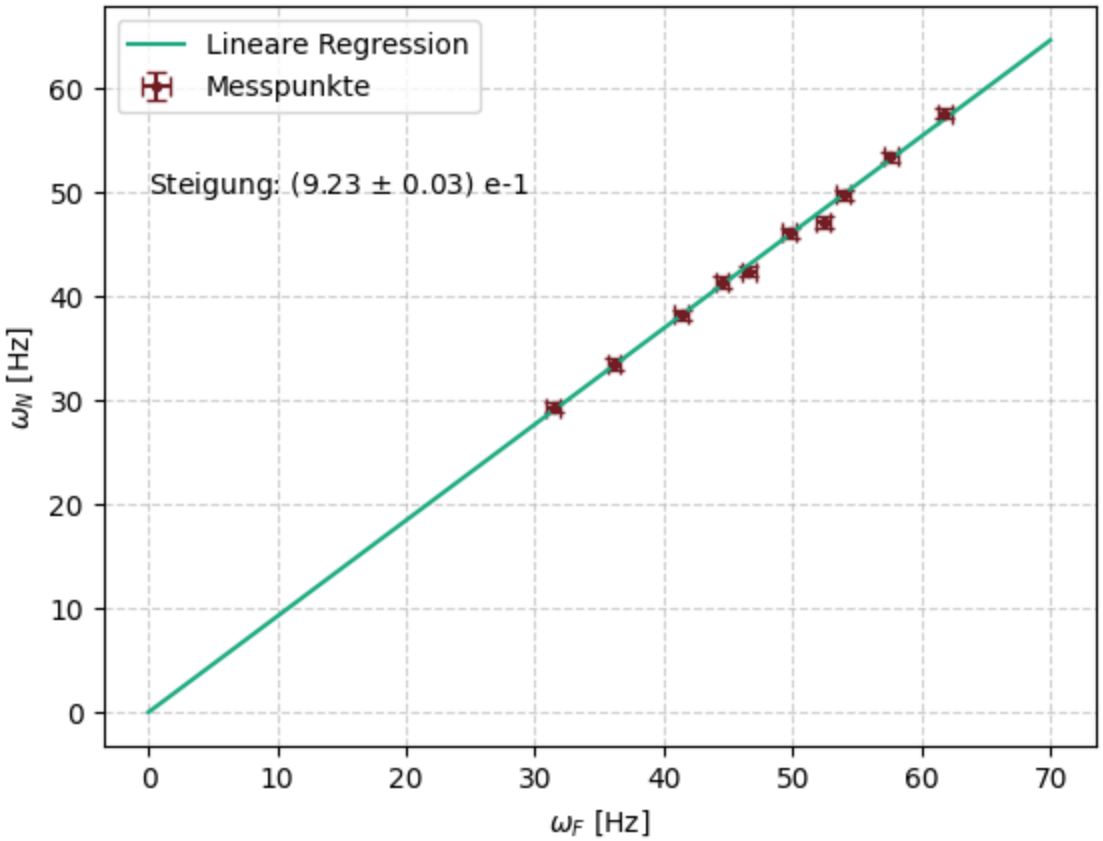


In [123…

```
aufgabe = "6_alt"

# Einheit in Hz
omega_n = np.array([550, 510, 475, 450, 440, 405, 395, 365, 320, 280]) * 2 * np.pi /60
omega_f_2 = np.array([590, 550, 515, 500, 475, 445, 425, 395, 345, 300]) * 2 * np.pi /60

err_omega = 5 * 2 * np.pi /60

# ------------------------
#   Erstellung des Plots
# ------------------------

# Durch den Ursprung laufen lassen; Umsetzungsidee kam von 'Luca Hafner' :)
```

```python
def linear(x,a):
    return a*x
x = np.linspace(0, 70, 10000)

popt, pcov = curve_fit(linear, omega_f_2, omega_n, sigma=err_omega, absolute_sigma=True)

steig_2 = round_sig_digs(popt[0], np.sqrt(pcov[0,0]))
#absch_2 = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))

print(f"Steigung: {steig_2[2]}")


plt.errorbar(
    omega_f_2,
    omega_n,
    err_omega,
    err_omega,
    fmt='.',
    color=weinrot,
    capsize=3.5,
    label="Messpunkte")

plt.plot(
    x,
    linear(x,*popt),
    color=hellgruen,
    label="Lineare Regression")


plt.xlabel('$\\omega_F \\ [\\mathrm{Hz}]$')
plt.ylabel('$\\omega_N \\ [\\mathrm{Hz}]$')
plt.title('')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.text(0, 50, f"Steigung: {steig_2[2]}")

plt.savefig(f'{pfad}omegaN_gegen_omegaF{aufgabe}.pdf')
plt.show()
```

```
Steigung: (9.23 $\pm$ 0.03) e-1
```



Formeln

In [124…

```
Iz_s, m_s = sp.symbols('I_z, m')
# \\frac{\\omega_F}{\\omega_N}
params=[Iz_s, m_s]

Ix_2 = Iz_s * (1/m_s)

print(steig_2[2])

values = ([
    iz[0], iz[1],
    steig_2[0], steig_2[1]
])

calc2(Ix_2, params, values, [])
```

```
(9.23 $\pm$ 0.03) e-1
gegebene Funktion:
Die gegebene Funktion lautet:
```

$$\frac{I_z}{m}$$

```
Hier ist der dazugehörige Latex code:
```

```
\frac{I_{z}}{m}
```

Kopieren

```
Formel des absoluten Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:
```

$$\sqrt{\frac{1}{m^4}\left(I_z^2 \Delta m^2 + m^2 \Delta I_z^2\right)}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{1}{m^{4}} \left(I_{z}^{2} \Delta m^{2} + m^{2} \Delta I_{z}
^{2}\right)}
```

Kopieren

Formel des relativen Fehlers der gegebenen Funktion:
Die gegebene Funktion lautet:

$$\sqrt{\frac{\Delta m^2}{m^2} + \frac{\Delta I_z^2}{I_z^2}}$$

Hier ist der dazugehörige Latex code:

```
\sqrt{\frac{\Delta m^{2}}{m^{2}} + \frac{\Delta I_{z}^{2}}{I_{z}^{2}}}
```

Kopieren

Out[124…

```
array([[4.74327798, 0.12049799]])
```